

Report

Team members:

Rofida Ramadan 202201888

Mariam Samir 202200562

Ahmed Ezzat 202402481

Step 1: Data Loading

Loading data from CSV File

▼ Data Loading

This is formatted as code

[] import pandas as pd
import numpy as np

[] data=pd.read_csv('/content/data.csv')

data

	ad_id	type	price	area	bedrooms	bathrooms	level	furnished	rent	city	region
0	Ad id 194281336	apartment	3790000.0	207.0	3	3	2	NaN	no	giza	6th of october
1	Ad id 194198684	apartment	870000.0	155.0	3	3	4	no	no	cairo	new heliopolls
2	Ad id 191102186	apartment	1546400.0	170.0	3	2	9	NaN	no	cairo	zahraa al maadi
3	Ad id 193951860	apartment	1300000.0	270.0	4	3	ground	no	no	sharqia	10th of ramadan
4	Ad id 194064010	duplex	5888000.0	387.0	4	4	ground	no	no	cairo	new cairo - el tagamoa
...	...	...	...	...	...	...	...	...	...	...	...
54324	Ad id 193833696	apartment	18000.0	220.0	2	3	2	yes	yes	alexandria	kafr abdo

Step 2: Data Cleaning

Handling Missing & Invalid Entries:

▼ Data Cleaning

▶ data.info()

Show hidden output

[] data.isnull().sum()

Show hidden output

[] duplicates = data[data.duplicated()]
duplicates

Show hidden output

Next steps: [Generate code with duplicates](#) [View recommended plots](#) [New interactive sheet](#)

[] duplicates = data[data.duplicated()]
print(duplicates)

Show hidden output

[] sample_duplicates = duplicates.sample(n=10)
sample_duplicates

▶ duplicate=data_cleaned.duplicated()
duplicate.sum()

Show hidden output

[] np.int64(0)

[] data.shape

Show hidden output

[] data_cleaned.dropna()

Show hidden output

[] data_cleaned.isnull().sum()

Show hidden output

[] data_cleaned['furnished'] = data_cleaned['furnished'].str.strip()

Show hidden output

[] data_cleaned.replace("", np.nan, inplace=True)
data_cleaned.replace("None", np.nan, inplace=True)
data_cleaned.dropna(inplace=True)

Other actions:

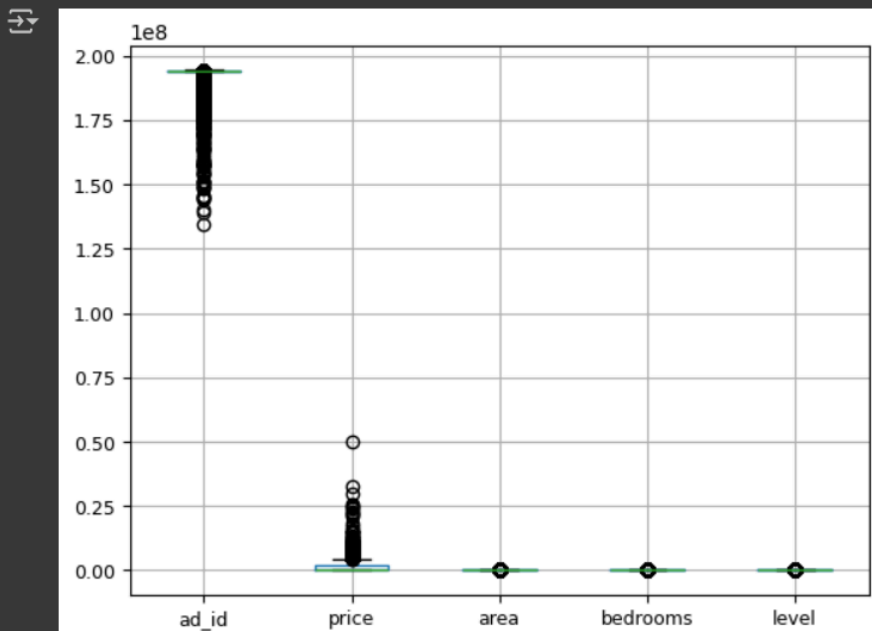
- All blank strings (" ") and "None" entries were replaced with `np.nan`.
- Final removal of `NaN` values from critical column

Outlier Handling

- Used **boxplots** to visualize numeric columns.
- Applied **clipping** to the **price** column (5th–95th percentile)
- Re-visualized boxplots after clipping.

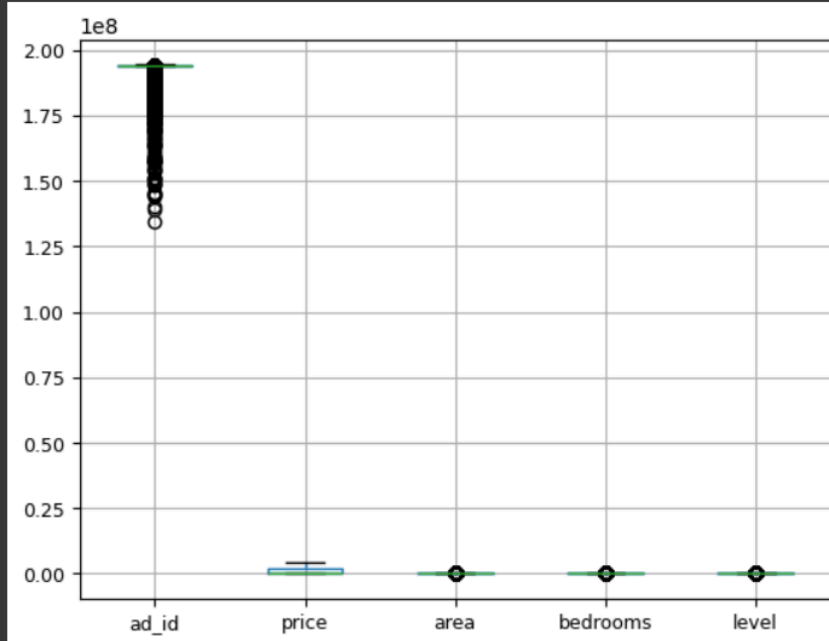
Before Check Outlier:

```
[ ] numerical_columns=['ad_id','price','area','bedrooms','level']  
  
boxplot = data_cleaned.boxplot(column=numerical_columns, grid=True, fontsize=9)  
plt.show()
```



After Remove Outliers:

```
[ ] boxplot =data_cleaned.boxplot(column=numerical_columns, grid=True, fontsize=9)  
plt.show()
```



. Feature Transformation

- Converted appropriate columns to **int** or **float**.
- Standardized all column names by stripping spaces.
- Ensured **furnished**, **rent**, and **type** are within allowed categorical values.

```
[ ] data_cleaned['level'].replace(["", "None"], np.nan, inplace=True)
data_cleaned.dropna(subset=['level'], inplace=True)
data_cleaned['bedrooms'].replace(["", "None"], np.nan, inplace=True)
data_cleaned.dropna(subset=['bedrooms'], inplace=True)

[ ] data_cleaned.isnull().sum()

[ ] data_cleaned['type'].unique()

[ ] data_cleaned.info()

[ ] data_cleaned['bathrooms'].unique()

data_cleaned['bathrooms'] = data_cleaned['bathrooms'].replace({'10+': 10}).astype(int)
```

```
data_cleaned['bedrooms'] = data_cleaned['bedrooms'].replace({'10+': 10}).astype(int)

[ ] data_cleaned = data_cleaned[data_cleaned['bathrooms'] >= 1]

[ ] data_cleaned['ad_id'] = data_cleaned['ad_id'].str.extract(r'(\d+)').astype(int)

[ ] # Remove rows with null values in 'furnished'
data_cleaned = data_cleaned.dropna(subset=['furnished'])

# Or, impute missing values with a suitable value (e.g., 'no')
data_cleaned['furnished'] = data_cleaned['furnished'].fillna('no')

[ ] data_cleaned.isnull().sum()

[ ] data_cleaned.info()
```

- Handling the categorical values from the numerical columns by replacing it with numbers.

Step 3 : Data Validation Using Pandera

Defined and applied a `DataFrameSchema` using `pandora` to ensure

```
!pip install pandera
import pandera as pa
import pandas as pd
from pandera import Column, DataFrameSchema, Check, Index

[ ] import pandera as pa

schema = pa.DataFrameSchema({
    "price": pa.Column(float, pa.Check(lambda x: x >= 0)), # Price must be non-negative
    "area": pa.Column(float, pa.Check.in_range(1, 1000)), # Area range
    "bathrooms": pa.Column(int, pa.Check.in_range(1, 10)), # Bathrooms between 1 and 10
    "bedrooms": pa.Column(int, pa.Check(lambda x: x > 0)), # Bedrooms must be positive
    "ad_id": pa.Column(int, pa.Check(lambda x: x > 0)), # Ad ID must be valid
    "type": pa.Column(str, pa.Check.isin(["apartment", "duplex", "penthouse", "studio", "room"])), # Property types
    "level": pa.Column(float, pa.Check(lambda x: x >= 0)), # Level must be non-negative
    "furnished": pa.Column(str, pa.Check.isin(["yes", "no"])), # Only 'yes' or 'no' allowed
    "rent": pa.Column(str, pa.Check.isin(["yes", "no"])), # Only 'yes' or 'no' allowed
    "city": pa.Column(str), # Check if city names exist
    "region": pa.Column(str), # Ensure region is valid
    "level": pa.Column(int, pa.Check(lambda x: x >= 0))
})

[ ] validated_data = schema.validate(data_cleaned)
print(validated_data.head())
```

```
validated_data = schema.validate(data_cleaned)
print(validated_data.head())
```

	ad_id	type	price	area	bedrooms	bathrooms	level	\
1	194198684	apartment	870000.0	155.0	3	3	4	
3	193951860	apartment	1300000.0	270.0	4	3	0	
4	194064010	duplex	5888000.0	387.0	4	4	0	
5	194021448	apartment	950000.0	104.0	2	1	7	
6	194294715	apartment	2100000.0	160.0	3	2	1	

	furnished	rent	city	region
1	no	no	cairo	new heliopolis
3	no	no	sharqia	10th of ramadan
4	no	no	cairo	new cairo - el tagamoa
5	no	no	cairo	nasr city
6	no	no	cairo	mostakbal city

```
[ ] !pip install pandera[io]

import pandera as pa
from pandera import Column, DataFrameSchema, Check, Index
inferred_schema = pa.infer_schema(data_cleaned).to_script()
print(inferred_schema)

[ ] print(data_cleaned.dtypes)
```

. Automated Data Profiling After Cleaning

Tool: ydata-profiling

- Profile report generated with:
 - Data types
 - Missing values summary
 - Correlation matrix
 - Duplicates
 - Descriptive statistics

 Saved Output: [report.html](#)

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This
ydata-profiling 4.16.1 requires numpy<2.2,>=1.16.0, but you have numpy 2.2.4 which is incompatible.
numba 0.60.0 requires numpy<2.1,>=1.22, but you have numpy 2.2.4 which is incompatible.
tensorflow 2.18.0 requires numpy<2.1.0,>=1.26.0, but you have numpy 2.2.4 which is incompatible.
Successfully installed numpy-2.2.4
```

 !pip install ydata_profiling

 Show hidden output

```
[ ] from ydata_profiling import ProfileReport
import pandas as pd

profile = ProfileReport(data_cleaned, title="Data Profiling Report")
profile.to_file("report.html")
```

 Summarize dataset: 100%  56/56 [00:06<00:00, 7.40it/s, Completed]

0%		0/11	[00:00<?, ?it/s]
9%		1/11	[00:00<00:01, 9.21it/s]
27%		3/11	[00:00<00:00, 13.94it/s]
64%		7/11	[00:00<00:00, 24.62it/s]
100%		11/11	[00:00<00:00, 16.64it/s]

Generate report structure: 100%  1/1 [00:07<00:00, 7.60s/it]

Render HTML: 100%  1/1 [00:02<00:00, 2.70s/it]

Export report to file: 100%  1/1 [00:00<00:00, 11.13it/s]

Summary:

Dataset cleaned, validated, and ready for modeling or EDA.

- All fields are now with the suitable datatypes.
- Outliers managed, nulls removed, and structure validated.